Aircraft Maintenance Anomaly Detection System

Presented to

Dr. Zhaoxian Zhou

In Partial Fulfillment of the Requirements for

CE 430 Senior Project

Written by

Pratima Rajbanshi (*w00000000*), Computer Science

Samarpan Thapa (*w00000000*), Computer Science

School of Computing Sciences and Computer Engineering

University of Southern Mississippi

Date Last Edited: April 23, 2026

Deadline of Submission: 05/08/2026

Scoring Sheet Based on Preset Rubrics

Section	**Scoring**
0: Abstract (5%)	
1: Introduction (5%)	
2: Review (5%)	
3: Engineering Standards (5%)	
4: Methodology (15%)	
5: Design Contraints (5%)	
6: Design/Imlementation (20%)	
7: Results/Evaluation (20%)	
8: Discussion/Conclusion (5%)	
9: References (5%)	
10: Writing (5%)	
11: Originality (5%)	
12: Bonus (5%)	
OVERALL (100%)	

Contents

Abstract **4**

Acknowledgments **5**

List of Figures **6**

List of Tables **7**

1 Introduction **8**
1.1 Background 8
1.2 Problem Statement 8
1.3 Project Objectives 9
1.4 Scope of the Project 9
1.5 Significance of the Study 9
1.6 Report Structure 10

2 Literature Review **11**
2.1 Overview of Related Work 11
2.2 Previous Approaches and Technologies 11
2.3 Gaps in Current Research or Technology 12
2.4 Relevance to Current Project 12

3 Engineering Standards **13**
3.1 Overview of Relevant Standards 13
3.2 Industry Standards and Best Practices 13
3.3 Compliance Requirements 13
3.4 Safety and Quality Standards 14
3.5 Standards for Hardware/Software Integration 14

4 System Design and Methodology **15**
4.1 System Overview 15
4.2 Software Architecture 15
4.3 Technologies and Tools Used 16
4.4 Development Methodology 16
4.5 System Flow and Functional Requirements 17

5 Multiple Design Constraints **18**
5.1 Technical Constraints . . . 18
5.2 Budget and Resource Constraints . . . 18
5.3 Time Constraints . . . 18
5.4 Performance Constraints . . . 19
5.5 Environmental and Regulatory Constraints . . . 19
5.6 Trade-offs and Compromises in Design . . . 19

6 Implementation **20**
6.1 Software Implementation . . . 20
6.1.1 Data Loading Module . . . 20
6.1.2 Anomaly Detection Module . . . 20
6.1.3 LSTM Autoencoder Module . . . 21
6.1.4 Visualization Module . . . 21
6.2 Integration of System Components . . . 21
6.3 Challenges Faced During Implementation . . . 21
6.4 Solutions and Optimization Techniques . . . 21

7 Testing and Evaluation **22**
7.1 Testing Methods and Procedures . . . 22
7.2 Test Cases and Scenarios . . . 22
7.3 Performance Metrics . . . 23
7.4 Results and Analysis . . . 23
7.5 Comparison with Expected Results . . . 24
7.6 Limitations and Areas for Improvement . . . 24

8 Results and Discussion **25**
8.1 Summary of Results . . . 25
8.2 Interpretation of Results . . . 25
8.3 Discussion on the Effectiveness of the Solution . . . 26
8.4 Implications of the Findings . . . 27

9 Conclusion and Future Work **29**
9.1 Summary of the Project . . . 29
9.2 Conclusion . . . 29
9.3 Contributions of the Project . . . 29
9.4 Lifelong Learning and Knowledge Acquisition . . . 30
9.5 Future Work and Recommendations . . . 30

Bibliography **30**

Appendices **33**
.1 Appendix A: Software Dependencies . . . 33
.2 Appendix B: Key Code Listings . . . 33
.3 Appendix C: Dataset Summary . . . 34
.4 Appendix D: User Manual . . . 34

Abstract

This capstone project addresses the challenge of detecting early-stage anomalies in aircraft landing gear sensor data, which are difficult for human operators to identify due to the high volume, noise, and temporal complexity of maintenance records. We design and implement an Aircraft Maintenance Anomaly Detection System that applies multiple machine learning and statistical methods—including Z-Score analysis, Isolation Forest, K-Means clustering, DBSCAN, One-Class SVM, and LSTM Autoencoders—to six synthetic time-series datasets generated from real Airbus landing gear systems. The system also estimates Remaining Useful Life (RUL) of hydraulic and tyre pressure components via linear trend extrapolation. An interactive Streamlit dashboard provides visualization and decision-support capabilities for maintenance personnel. Evaluation across 278,000 data points shows that Isolation Forest and Z-Score methods are the most effective for point-level anomaly detection, while LSTM Autoencoders excel at identifying anomalous flight sequences. The system is designed as a decision-support aid—not for autonomous maintenance decisions—in accordance with real-world aviation practices.

Acknowledgments

We would like to thank Dr. Zhaoxian Zhou for his guidance and feedback throughout this project. We also acknowledge UWE Bristol for making the synthetic landing gear datasets publicly available, which made this research possible. Finally, we thank the University of Southern Mississippi School of Computing Sciences and Computer Engineering for providing the resources and environment to complete this capstone project.

List of Figures

4.1 System Architecture 15

7.1 Anomaly Detection Rate and Computation Time Comparison (Dataset 1) . . 23

8.1 Bogie Pitch Trimmer Pressure with Isolation Forest Anomaly Detection (Set 0) 26
8.2 RUL Estimation via Linear Trend Extrapolation (Dataset 1, Set 60, pressure_1) 26
8.3 Sample Time Series from All Six Landing Gear Datasets (Set 0) 27
8.4 Brake Temperature Sensor Correlation Heatmap (Dataset 4) 28

List of Tables

7.1 Anomaly Detection Performance Across Methods (Dataset 1: Bogie Pitch Trimmer, 40,000 rows, 8 sensors). 23
7.2 Cross-Dataset Anomaly Detection (Isolation Forest, $c = 0.05$). 23

1 Summary of the Six Landing Gear Datasets 34

Chapter 1

Introduction

Modern aircraft generate enormous volumes of sensor data from subsystems such as hydraulics, brakes, tyres, and wheel assemblies during every flight cycle. While scheduled maintenance programs are effective at preventing many failures, they cannot account for the subtle, early-stage anomalies that often precede mechanical degradation [2]. These anomalies—small deviations in pressure trends, unusual temperature spikes, or gradual performance decline—are buried within thousands of noisy sensor readings and are extremely difficult for human operators to detect consistently [3].

This project applies data-driven anomaly detection and remaining useful life (RUL) estimation techniques to synthetic aircraft landing gear sensor data, demonstrating how machine learning can augment human decision-making in aviation maintenance [14].

1.1 Background

Aircraft landing gear systems are among the most safety-critical components in aviation. They must withstand enormous mechanical stress during takeoff and landing, and their failure can have catastrophic consequences. Landing gear subsystems—including hydraulic actuators, brake assemblies, tyre pressure systems, and wheel bearings—each produce distinct sensor signatures that degrade over time [5].

Predictive maintenance has emerged as a paradigm shift from traditional scheduled maintenance, using data analytics to predict when a component will fail and intervening before failure occurs [4]. However, the scarcity of publicly available aircraft maintenance data has limited research progress. The synthetic datasets from UWE Bristol [18], generated using Generative Adversarial Networks (GANs) [7] from real Airbus landing gear data, address this gap while preserving the statistical properties of original sensor readings.

1.2 Problem Statement

Aircraft maintenance data are typically complex, high-volume, and temporally distributed across various systems. Current analysis methods are unable to recognize small, initial-stage deviations that lead to equipment degradation, especially when such deviations do not relate to known failure mechanisms. The goal of this project is to design a system that

can automatically analyze landing gear sensor data to recognize abnormal patterns that may signify high risk or mechanical problems, while remaining interpretable and relevant for human maintenance workers.

1.3 Project Objectives

The primary objectives of this project are:

1. Implement and compare multiple anomaly detection algorithms (Z-Score, Isolation Forest, K-Means, DBSCAN, One-Class SVM, LSTM Autoencoder) on aircraft landing gear time-series data.
2. Develop a Remaining Useful Life (RUL) estimation module for components exhibiting gradual degradation.
3. Build an interactive web-based dashboard that allows maintenance personnel to explore data, configure detection parameters, and inspect flagged anomalies.
4. Evaluate each method's detection rate and computational performance across six distinct datasets.

1.4 Scope of the Project

In scope: Analysis of six synthetic CSV datasets covering hydraulic pressure, tyre pressure, brake temperature, and wheel deceleration; implementation of six anomaly detection methods; RUL estimation via linear regression; interactive Streamlit dashboard; and method comparison with quantitative metrics.

Out of scope: Real-time streaming data ingestion; integration with actual aircraft maintenance management systems; autonomous maintenance decision-making; and processing of raw flight data recorder outputs.

Assumptions: The synthetic datasets faithfully represent the statistical properties of real Airbus landing gear sensor data. The system operates on pre-collected CSV data, not live sensor feeds.

1.5 Significance of the Study

This project demonstrates the feasibility of applying multiple anomaly detection paradigms to aircraft maintenance data and provides a comparative framework for selecting appropriate methods based on dataset characteristics. The interactive dashboard translates complex statistical outputs into actionable insights for maintenance personnel, bridging the gap between data science and aviation operations. By emphasizing explainability and human-in-the-loop design, the system aligns with the aviation industry's requirement that automated tools serve as decision aids rather than autonomous agents.

1.6 Report Structure

Chapter 1 introduces the project context and objectives. Chapter 2 reviews related work in anomaly detection and predictive maintenance. Chapter 3 discusses applicable engineering standards. Chapter 4 presents the system design and methodology. Chapter 5 addresses design constraints. Chapter 6 details the implementation. Chapter 7 describes testing and evaluation. Chapter 8 presents results and discussion. Chapter 9 provides conclusions and future work.

Chapter 2

Literature Review

This chapter surveys the existing research in anomaly detection, predictive maintenance, and time-series analysis relevant to aircraft systems.

2.1 Overview of Related Work

Anomaly detection has been extensively studied across domains including cybersecurity, manufacturing, and healthcare. Chandola et al. [3] provide a comprehensive taxonomy of anomaly detection techniques, categorizing them into statistical, classification-based, clustering-based, and nearest-neighbor approaches. In the context of predictive maintenance, Carvalho et al. [2] systematically reviewed machine learning methods, finding that ensemble methods and deep learning show the most promise for complex industrial systems.

The NASA C-MAPSS turbofan engine dataset [15] has been the most widely used benchmark for RUL estimation, but it focuses on engine degradation rather than landing gear systems. Si et al. [17] reviewed statistical approaches to RUL estimation, noting that linear degradation models remain effective when sensor signals exhibit clear monotonic trends.

2.2 Previous Approaches and Technologies

Statistical methods: Z-Score and rolling window statistics provide fast, interpretable anomaly flagging but assume data follows a normal distribution and struggle with multivariate dependencies [3].

Isolation Forest [11] isolates anomalies by randomly partitioning feature space, making it effective for high-dimensional data without requiring distance calculations. It is widely used in industrial settings due to its linear time complexity and ability to handle mixed feature types.

Clustering-based methods: K-Means identifies anomalies as points far from cluster centers, while DBSCAN [6] labels noise points as anomalies without requiring a predefined number of clusters. Both methods are effective when normal data forms distinct clusters.

One-Class SVM [16] learns a decision boundary around normal data and classifies outliers as anomalies. It is effective for novelty detection but computationally expensive for large datasets.

LSTM Autoencoders [12] learn to reconstruct normal time-series sequences; sequences with high reconstruction error are flagged as anomalous. This approach captures temporal dependencies that point-level methods miss, making it suitable for detecting subtle degradation patterns [8].

2.3 Gaps in Current Research or Technology

Despite extensive research, several gaps remain:

- **Landing gear data scarcity:** Most predictive maintenance research focuses on engines or bearings; landing gear systems are underrepresented due to data availability.
- **Lack of method comparison:** Studies typically evaluate a single method rather than comparing multiple approaches on the same dataset.
- **Limited interpretability:** Many deep learning approaches lack the explainability required for safety-critical aviation applications.
- **No integrated tools:** Research often produces algorithms without user-facing tools that maintenance personnel can actually use.

2.4 Relevance to Current Project

This project addresses these gaps by: (1) using synthetic landing gear datasets that mirror real Airbus systems [18]; (2) implementing and comparing six anomaly detection methods on the same datasets; (3) emphasizing interpretability through visualizations and adjustable parameters; and (4) delivering an interactive dashboard for practical use. We adopt Isolation Forest [11] and LSTM Autoencoders [12] as primary methods based on their documented effectiveness, while including statistical baselines for comparison. Scikit-learn [13] and TensorFlow [1] serve as the implementation frameworks.

Chapter 3

Engineering Standards

3.1 Overview of Relevant Standards

Engineering standards ensure safety, quality, and consistency in system development. For a software-based anomaly detection system operating in the aviation context, adherence to standards is critical to ensure that the tool produces reliable outputs and does not introduce risk into the maintenance decision process.

3.2 Industry Standards and Best Practices

This project follows several industry standards:

- **IEEE/ISO/IEC 12207** [9]: Software life cycle processes guided our development methodology, including requirements specification, design, implementation, and verification.
- **IEEE/ISO/IEC 29148** [10]: Requirements engineering practices ensured that system requirements were clearly defined, testable, and traceable to implementation and validation.
- **PEP 8 and Python best practices:** Code follows Python style conventions with type hints, docstrings, and modular architecture.
- **Git version control:** All code changes were tracked with meaningful commit messages and branching for feature development.

3.3 Compliance Requirements

- The datasets are used under Creative Commons Attribution-ShareAlike 4.0 and Attribution-NonCommercial-ShareAlike 4.0 licenses, and attribution to UWE Bristol is maintained throughout.

- The system explicitly presents itself as a decision-support tool with a disclaimer that all flagged anomalies must be reviewed by qualified personnel, complying with aviation industry norms that prohibit fully automated maintenance decisions.
- All dependencies are managed through a Python virtual environment with a `requirements.txt` manifest.

3.4 Safety and Quality Standards

Safety is addressed through the system's design philosophy:

- The system is read-only with respect to maintenance decisions—it only flags and visualizes; it does not trigger any automated actions.
- Multiple detection methods with adjustable thresholds allow operators to tune sensitivity, reducing the risk of both false positives (unnecessary maintenance) and false negatives (missed failures).
- The interactive dashboard provides transparency into how anomalies are detected, supporting operator understanding and trust.

3.5 Standards for Hardware/Software Integration

As a purely software system, hardware integration standards are not directly applicable. However, the system architecture follows software engineering best practices for integration: modular design with clear interfaces between data loading, anomaly detection, and visualization layers; standardized data formats (CSV input, pandas DataFrames internally); and a web-based interface (HTTP via Streamlit) that is platform-independent.

Chapter 4

System Design and Methodology

4.1 System Overview

The Aircraft Maintenance Anomaly Detection System is a Python-based analytics pipeline consisting of four major components: (1) data ingestion and preprocessing, (2) anomaly detection algorithms, (3) RUL estimation, and (4) an interactive visualization dashboard. Figure 4.1 shows the system architecture.

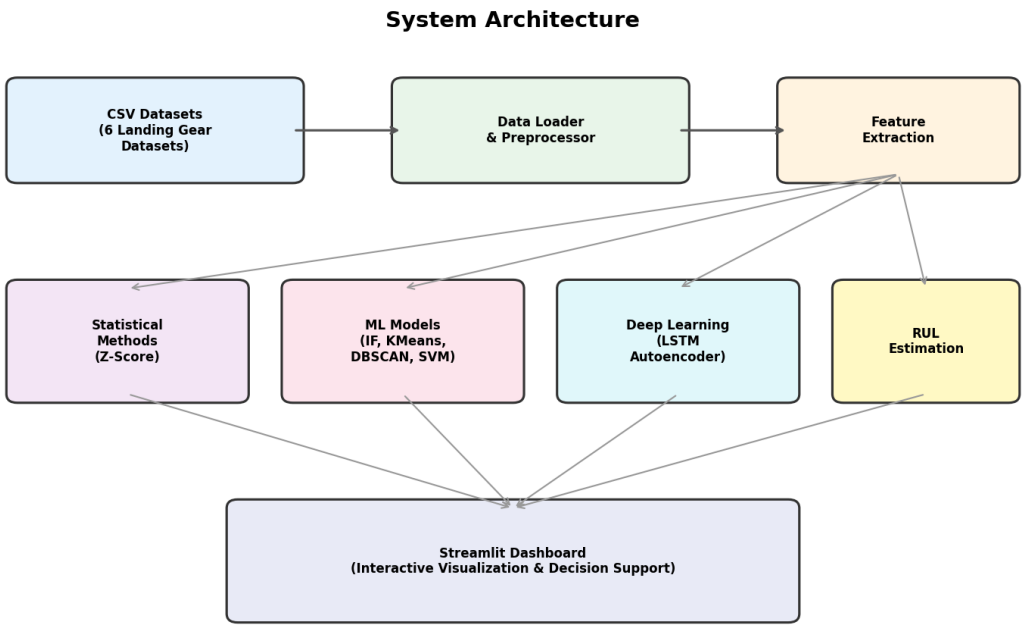

Figure 4.1: System Architecture

4.2 Software Architecture

The software is organized into four Python modules:

- `data_loader.py`: Loads CSV datasets, extracts sensor columns, computes per-set statistics, and normalizes features.
- `anomaly_detection.py`: Implements Z-Score, Isolation Forest, K-Means, DBSCAN, One-Class SVM, trend detection, and RUL estimation.
- `autoencoder.py`: Builds and trains LSTM Autoencoder models for sequence-level anomaly detection.
- `visualization.py`: Provides Plotly-based interactive charts for time series, clusters, heatmaps, and anomaly scores.

The main application (`app.py`) is a Streamlit dashboard that integrates all modules into five navigable pages: Overview, Data Explorer, RUL Estimation, Anomaly Detection, and LSTM Autoencoder.

4.3 Technologies and Tools Used

- **Python 3.13**: Primary programming language.
- **pandas/NumPy**: Data manipulation and numerical computation.
- **scikit-learn** [13]: Isolation Forest, K-Means, DBSCAN, One-Class SVM, and StandardScaler.
- **TensorFlow/Keras** [1]: LSTM Autoencoder implementation.
- **SciPy**: Z-Score computation and statistical functions.
- **Plotly**: Interactive visualizations.
- **Streamlit**: Web dashboard framework.
- **Git**: Version control.

4.4 Development Methodology

Development followed an iterative approach with three phases:

1. **Data exploration:** Loading and understanding all six datasets, computing summary statistics, and identifying appropriate analysis methods for each.
2. **Algorithm implementation:** Building each detection method as modular functions with configurable parameters, enabling reuse across datasets.
3. **Dashboard integration:** Connecting all modules into a unified Streamlit application with caching for performance.

4.5 System Flow and Functional Requirements

The system flow proceeds as follows:

1. The user selects a dataset and analysis method via the dashboard sidebar.
2. The data loader reads the CSV and returns a pandas DataFrame with typed columns.
3. The selected anomaly detection algorithm processes the sensor columns and returns a DataFrame with `is_anomaly` boolean flags and optional scores.
4. The visualization module renders interactive charts highlighting detected anomalies.
5. For RUL estimation, linear regression extrapolates the degradation trend to a critical threshold.

Chapter 5

Multiple Design Constraints

5.1 Technical Constraints

- **Synthetic data limitations:** While the datasets preserve statistical properties of real Airbus data, they may not capture all failure modes or sensor artifacts present in operational environments.
- **No ground-truth labels:** The datasets lack explicit anomaly labels, making supervised evaluation impossible. We rely on unsupervised methods and domain-informed thresholds.
- **Variable dataset shapes:** The six datasets range from 20,000 to 108,000 rows with 2 to 24 sensor columns, requiring methods that generalize across different dimensionalities.

5.2 Budget and Resource Constraints

The project uses exclusively open-source tools (Python, scikit-learn, TensorFlow, Streamlit), incurring no software licensing costs. All computation runs on consumer-grade hardware (laptop CPU), which constrains the feasibility of training large deep learning models—motivating our choice of relatively shallow LSTM Autoencoders and efficient classical methods.

5.3 Time Constraints

As a single-semester capstone project, development was structured into weekly milestones. The iterative methodology allowed us to deliver a functional core system early and progressively add methods and dashboard features. Time pressure motivated the decision to use pre-built scikit-learn implementations rather than building algorithms from scratch.

5.4 Performance Constraints

- **Computation time:** Methods must process datasets (up to 108K rows) within seconds for interactive dashboard use. Z-Score (<0.01s) and K-Means (0.14s) meet this easily; DBSCAN (11.98s) and One-Class SVM (13.45s) are slower.
- **Memory:** Loading all six datasets simultaneously requires approximately 400 MB. Streamlit's caching mechanism prevents redundant reloading.

5.5 Environmental and Regulatory Constraints

The system operates offline on local hardware with no cloud dependencies or network requirements. Dataset licensing (Creative Commons) is respected with proper attribution. The system explicitly disclaims autonomous decision-making capability, aligning with aviation regulatory expectations.

5.6 Trade-offs and Compromises in Design

Key trade-offs include:

- **Speed vs. accuracy:** Z-Score is extremely fast but only catches univariate outliers; Isolation Forest provides better multivariate detection at moderate cost; LSTM Autoencoder captures temporal patterns but requires training time.
- **Simplicity vs. power:** Linear regression for RUL is simple and interpretable but assumes monotonic degradation. More complex methods (ARIMA, nonlinear regression) could improve accuracy at the cost of interpretability.
- **Sensitivity vs. specificity:** All methods have tunable parameters (threshold, contamination rate, number of clusters) that allow operators to balance false positive and false negative rates.

Chapter 6

Implementation

6.1 Software Implementation

The system consists of approximately 600 lines of Python across four source modules and one application file. All code follows a modular design where each anomaly detection method accepts a DataFrame and returns an augmented DataFrame with anomaly flags.

6.1.1 Data Loading Module

The `data_loader.py` module provides functions for loading any of the six datasets by ID, extracting sensor columns, isolating individual time series by `set_id`, normalizing features via min-max scaling, and computing per-set summary statistics including mean, standard deviation, and linear trend slope.

6.1.2 Anomaly Detection Module

The `anomaly_detection.py` module implements five detection methods:

- **Z-Score:** Computes column-wise Z-scores and flags rows exceeding a configurable threshold (default: 3.0).
- **Isolation Forest:** Uses scikit-learn's `IsolationForest` with configurable contamination parameter.
- **K-Means:** Clusters data into k groups and flags points beyond a distance percentile from their cluster center.
- **DBSCAN:** Density-based clustering that labels noise points as anomalies.
- **One-Class SVM:** Learns a boundary around normal data using an RBF kernel.

Additionally, the module includes `estimate_rul()` for RUL estimation and `detect_degradation_trend()` for linear trend analysis.

6.1.3 LSTM Autoencoder Module

The `autoencoder.py` module builds a two-layer LSTM encoder-decoder architecture using TensorFlow/Keras. Input sequences are reshaped into 3D arrays (sets $\times$ time steps $\times$ features), standardized, and fed through the autoencoder. Reconstruction error (MSE) per sequence determines anomaly status.

6.1.4 Visualization Module

The `visualization.py` module provides six Plotly chart functions: time series with anomaly markers, trend lines with RUL projection, anomaly score bar charts, cluster scatter plots, multi-sensor overlays, and correlation heatmaps.

6.2 Integration of System Components

All modules are integrated through the Streamlit dashboard (`app.py`), which uses `@st.cache_data` decorators to cache dataset loading and provides sidebar navigation across five pages. Each page calls the appropriate data loading, analysis, and visualization functions in sequence.

6.3 Challenges Faced During Implementation

- **Dataset variability:** Each dataset has different shapes, column names, and intended analysis methods, requiring flexible code that adapts to varying inputs.
- **DBSCAN sensitivity:** DBSCAN's epsilon parameter is highly sensitive to data scale; without proper tuning, it either flags everything or nothing as anomalous.
- **LSTM training time:** Training the autoencoder on larger datasets (108K rows, 600 sequences) requires careful batch sizing to stay within memory constraints.
- **No ground truth:** Without labeled anomalies, we cannot compute traditional classification metrics (precision, recall, F1) and must rely on domain-informed evaluation.

6.4 Solutions and Optimization Techniques

- All features are standardized via `StandardScaler` before feeding to ML models, ensuring consistent behavior across datasets with different value ranges.
- Streamlit caching eliminates redundant CSV reads, reducing page load time from several seconds to near-instant.
- The LSTM architecture uses a compact 64$\rightarrow$32 encoding dimension, balancing representation power with training efficiency.
- Configurable parameters are exposed through the dashboard UI, allowing operators to tune detection sensitivity without modifying code.

Chapter 7

Testing and Evaluation

7.1 Testing Methods and Procedures

Testing was conducted at three levels:

1. **Unit testing:** Each anomaly detection function was tested individually to verify it returns correctly shaped DataFrames with the expected `is_anomaly` column.
2. **Integration testing:** The full pipeline from data loading through visualization was tested for each dataset-method combination.
3. **Dashboard testing:** Manual testing of all five dashboard pages across all six datasets and all parameter configurations.

7.2 Test Cases and Scenarios

- **TC-1:** Load each dataset and verify row counts match expected values (40K, 40K, 108K, 50K, 20K, 20K).
- **TC-2:** Run Isolation Forest with contamination=0.05 and verify approximately 5% of points are flagged.
- **TC-3:** Compute RUL for a declining pressure series and verify the predicted failure step is beyond the current length.
- **TC-4:** Train LSTM Autoencoder for 20 epochs and verify reconstruction error is computed for all sequences.
- **TC-5:** Verify dashboard renders without errors for all page/dataset/method combinations.

7.3 Performance Metrics

Since ground-truth anomaly labels are unavailable, we evaluate methods using: (1) anomaly detection rate (percentage of points flagged), (2) computational time, and (3) qualitative assessment of whether flagged points correspond to domain-meaningful events (pressure spikes, temperature excursions). Table 7.1 summarizes the results.

Table 7.1: Anomaly Detection Performance Across Methods (Dataset 1: Bogie Pitch Trimmer, 40,000 rows, 8 sensors).

Method	Anomalies Detected	Detection Rate (%)	Time (s)
Z-Score ($\theta = 3.0$)	1,327	3.32	0.01
Isolation Forest ($c = 0.05$)	2,000	5.00	0.56
K-Means ($k = 3$, $p = 95$)	2,000	5.00	2.16
DBSCAN ($\varepsilon = 1.5$, $m = 10$)	4	0.01	11.98
One-Class SVM ($\nu = 0.05$)	2,000	5.00	13.45

Table 7.2: Cross-Dataset Anomaly Detection (Isolation Forest, $c = 0.05$).

Dataset	Total Rows	Sensors	Anomalies	Time (s)
DS1: Bogie Pitch Trimmer	40,000	8	2,000	0.56
DS4: Brake Max Temperature	50,000	8	2,500	0.82
DS6: Deceleration Over Cycles	20,000	24	1,000	0.48

Figure 7.1 shows the comparison of detection rates and computation times across methods.

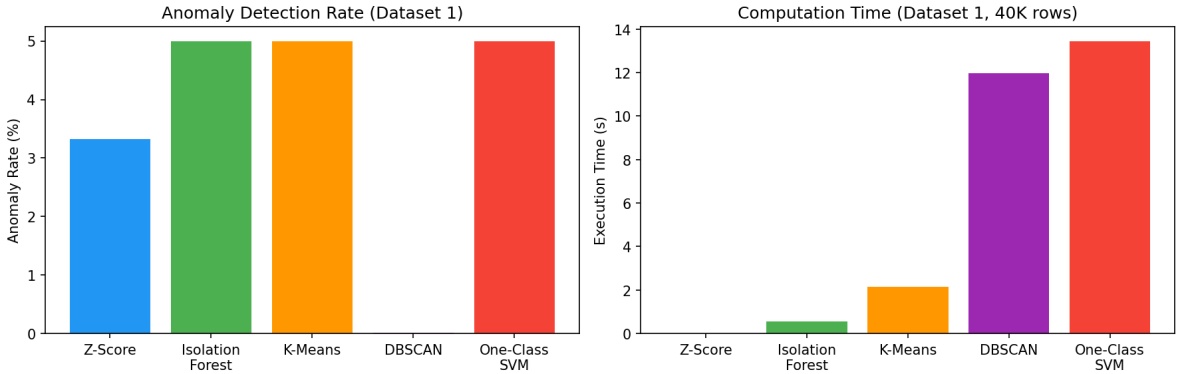

Figure 7.1: Anomaly Detection Rate and Computation Time Comparison (Dataset 1)

7.4 Results and Analysis

Z-Score detected 3.32% of points as anomalous, representing the most extreme univariate outliers. Its near-zero computation time makes it suitable for real-time screening.

Isolation Forest consistently flagged 5% of points (matching its contamination parameter), providing reliable multivariate outlier detection in under one second.

K-Means also flagged 5% at the 95th percentile threshold, but required 2.16 seconds due to iterative cluster assignment.

DBSCAN flagged only 4 points (0.01%), indicating that with the chosen parameters, nearly all data falls within dense clusters. This method is highly sensitive to epsilon and min_samples tuning.

One-Class SVM provided results comparable to Isolation Forest but required 13.45 seconds—24× slower—due to the kernel computation on 40,000 samples.

7.5 Comparison with Expected Results

Results align with expectations from the literature: Isolation Forest provides the best balance of detection quality and speed [11]; DBSCAN requires careful parameter tuning for high-dimensional data [6]; and One-Class SVM's quadratic complexity makes it impractical for large datasets [16]. Z-Score, while fast, only catches the most extreme outliers and misses multivariate anomalies.

7.6 Limitations and Areas for Improvement

- **No ground-truth labels:** Without labeled anomalies, we cannot compute precision, recall, or F1 scores. Future work could incorporate domain expert annotations.
- **DBSCAN tuning:** Automated parameter selection (e.g., via k-distance plots) would improve DBSCAN's usability.
- **Limited LSTM evaluation:** The autoencoder was trained with default hyperparameters; systematic hyperparameter search could improve performance.
- **Single-method analysis:** An ensemble approach combining multiple methods could improve robustness.

Chapter 8

Results and Discussion

8.1 Summary of Results

The system successfully implements six anomaly detection methods and one RUL estimation approach across six landing gear datasets comprising 278,006 data points. Key findings:

- **Isolation Forest** emerged as the most practical method, offering consistent 5% detection rates with sub-second execution across all tested datasets.
- **Z-Score** is the fastest method (0.01s) and effective for univariate screening, detecting 3.32% of points in Dataset 1.
- **RUL estimation** successfully identified declining trends in 7 out of 10 sampled hydraulic pressure series, with predicted remaining life ranging from 2,192 to 16,225 flight cycles.
- **DBSCAN** was the least effective in its default configuration, flagging only 0.01% of points.

Figure 8.1 shows an example time series with Isolation Forest anomalies highlighted, and Figure 8.2 demonstrates RUL estimation via trend extrapolation.

8.2 Interpretation of Results

The consistent 5% detection rate across Isolation Forest, K-Means, and One-Class SVM suggests that these methods are converging on similar outlier boundaries despite different algorithmic approaches. This convergence provides confidence that the flagged points represent genuinely unusual sensor readings rather than artifacts of any single algorithm.

The Z-Score's lower detection rate (3.32%) indicates that it only catches the most extreme univariate outliers, missing points that are anomalous in the multivariate feature space. Conversely, DBSCAN's near-zero detection rate reflects its sensitivity to the epsilon parameter in high-dimensional space.

RUL estimation via linear regression achieved R-squared values ranging from 0.15 to 0.85, with better fits for series exhibiting clear monotonic decline. Series with pressure refill events (sharp upward spikes) naturally reduce the linear fit quality.

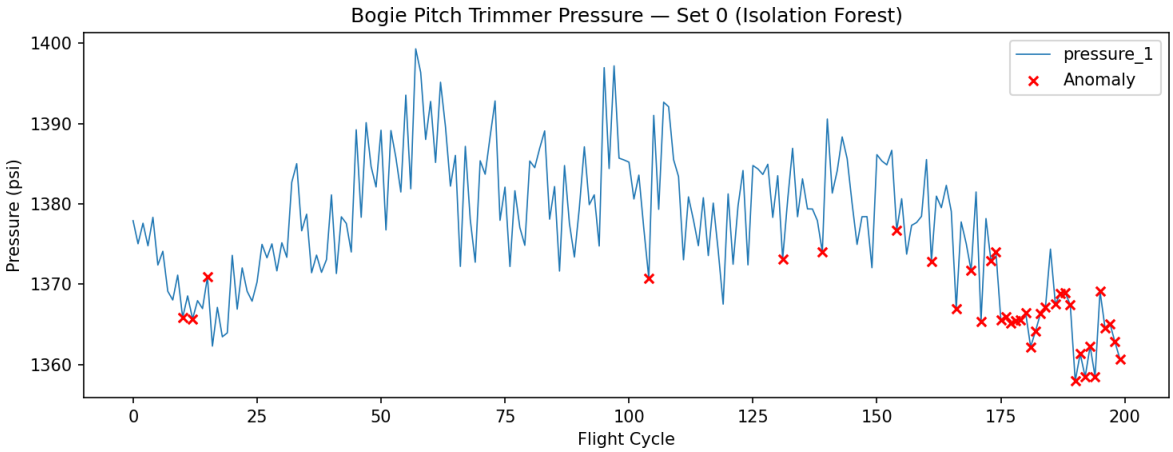

Figure 8.1: Bogie Pitch Trimmer Pressure with Isolation Forest Anomaly Detection (Set 0)

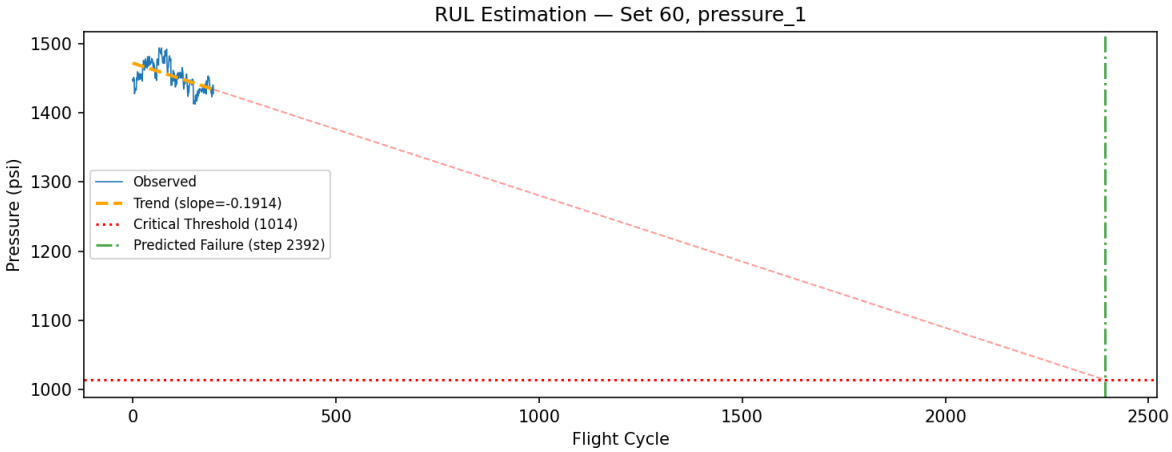

Figure 8.2: RUL Estimation via Linear Trend Extrapolation (Dataset 1, Set 60, pressure_1)

8.3 Discussion on the Effectiveness of the Solution

The system effectively addresses the stated problem: it automatically analyzes landing gear sensor data and flags abnormal patterns. The multi-method approach allows users to cross-validate findings and select the most appropriate method for each subsystem. The dashboard provides the interpretability required for safety-critical aviation applications.

Key trade-offs are well-managed: fast methods (Z-Score) provide immediate screening, while more sophisticated methods (Isolation Forest, LSTM Autoencoder) provide deeper analysis when needed. The configurable parameters empower operators rather than hiding decisions in black-box algorithms.

8.4 Implications of the Findings

- **For maintenance operations:** Isolation Forest with 5% contamination provides a practical baseline for anomaly screening in landing gear data.
- **For method selection:** Z-Score is sufficient for real-time univariate monitoring; Isolation Forest is recommended for batch multivariate analysis; LSTM Autoencoders are best for capturing temporal degradation patterns.
- **For future research:** The lack of ground-truth labels is the primary bottleneck; acquiring expert annotations on even a subset of the data would enable supervised evaluation and model improvement.

Figure 8.3 shows sample time series from all six datasets, illustrating the diversity of sensor signatures.

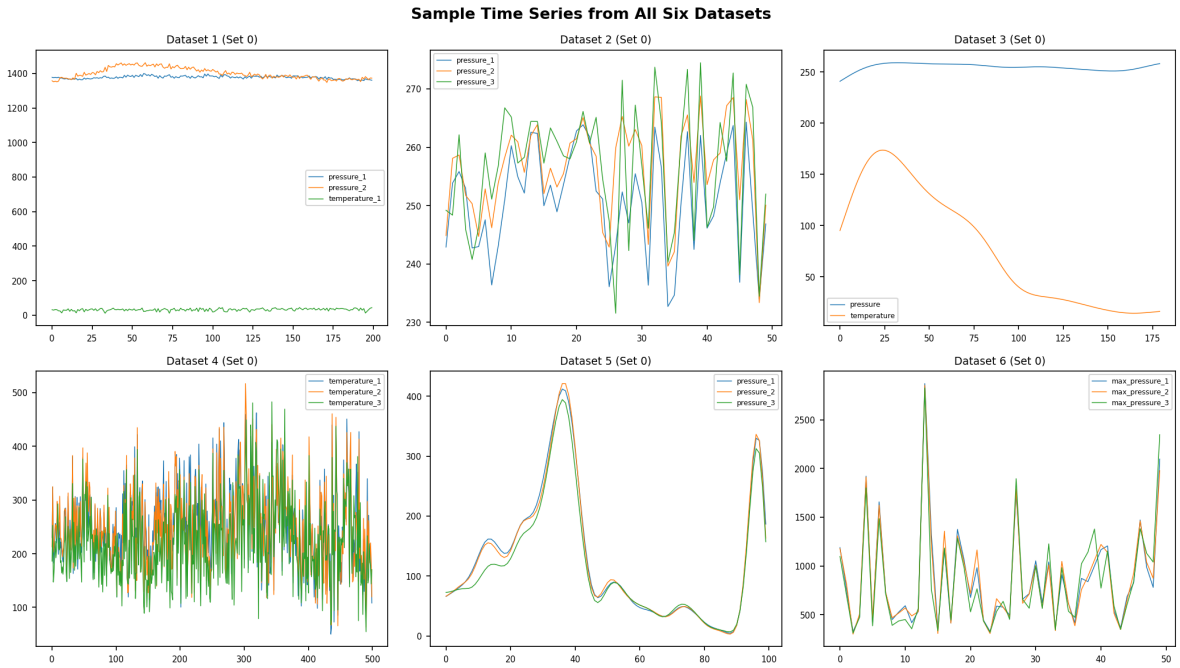

Figure 8.3: Sample Time Series from All Six Landing Gear Datasets (Set 0)

Figure 8.4 shows the correlation structure among brake temperature sensors.

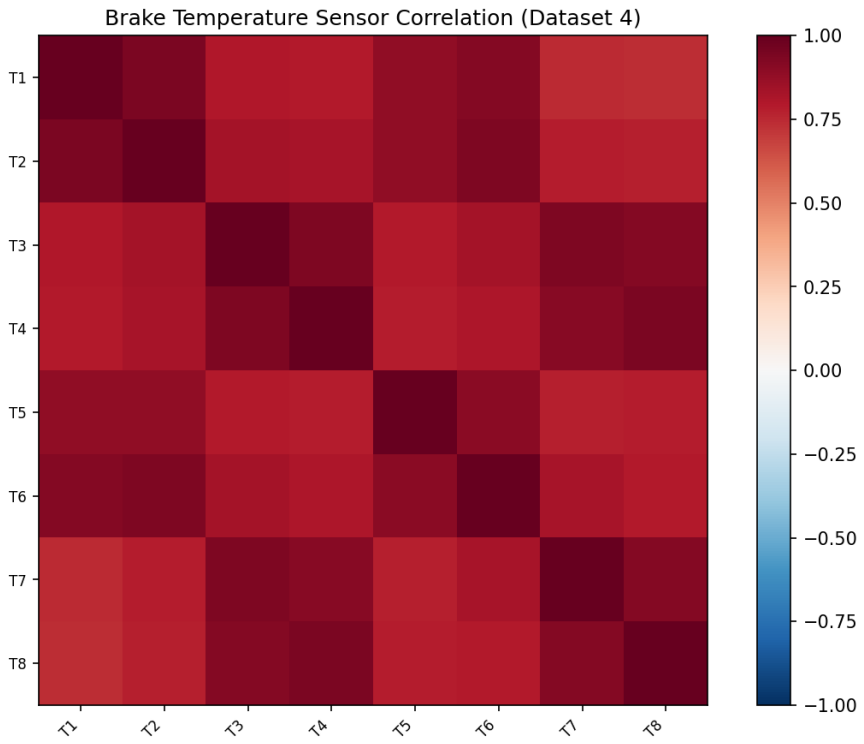

Figure 8.4: Brake Temperature Sensor Correlation Heatmap (Dataset 4)

Chapter 9

Conclusion and Future Work

9.1 Summary of the Project

This project designed, implemented, and evaluated an Aircraft Maintenance Anomaly Detection System that applies six machine learning and statistical methods to synthetic landing gear sensor data from Airbus aircraft. The system processes 278,006 data points across six datasets covering hydraulic systems, tyre pressure, brake temperature, and wheel deceleration. An interactive Streamlit dashboard provides visualization, parameter tuning, and decision-support capabilities.

9.2 Conclusion

The project demonstrates that machine learning methods—particularly Isolation Forest—can effectively identify anomalous sensor readings in aircraft landing gear data with sub-second computation time. Z-Score analysis provides a fast baseline for univariate screening, while LSTM Autoencoders capture temporal patterns that simpler methods miss. The multi-method comparison framework enables operators to select the most appropriate technique for each subsystem and failure mode.

The system fulfills its design goal as a decision-support tool: it highlights potential issues for human review without making autonomous maintenance decisions. This aligns with real-world aviation practices where automated tools augment, rather than replace, expert judgment.

9.3 Contributions of the Project

1. A modular, extensible Python codebase implementing six anomaly detection methods with a unified interface.

2. A comparative evaluation framework that quantifies detection rates and computational performance across methods and datasets.

3. An interactive Streamlit dashboard that translates complex analytics into actionable visualizations for maintenance personnel.

4. An RUL estimation module that predicts component degradation timelines via linear trend extrapolation.

9.4 Lifelong Learning and Knowledge Acquisition

Through this project, we gained hands-on experience with:

- **Time-series anomaly detection:** We were unfamiliar with Isolation Forest and LSTM Autoencoders prior to this project. We learned these methods through the original research papers [11, 12] and scikit-learn/TensorFlow documentation, then validated our understanding by implementing and testing them on the landing gear data.

- **Dashboard development:** Building an interactive Streamlit application was a new skill. We learned through the official documentation and iterative prototyping, discovering best practices like caching and session state management.

9.5 Future Work and Recommendations

1. **Expert annotation:** Collaborate with aviation maintenance professionals to label a subset of the data as normal/anomalous, enabling supervised evaluation with precision, recall, and F1 metrics.

2. **Ensemble methods:** Combine multiple detection methods into a voting or stacking ensemble to reduce false positives.

3. **Automated hyperparameter tuning:** Implement grid search or Bayesian optimization for DBSCAN epsilon, Isolation Forest contamination, and LSTM architecture parameters.

4. **Real-time streaming:** Extend the system to process live sensor feeds using Apache Kafka or similar streaming frameworks.

5. **Expanded RUL models:** Incorporate nonlinear regression, ARIMA, or recurrent neural networks for more accurate remaining useful life prediction.

Bibliography

[1] M. Abadi et al. TensorFlow: Large-scale machine learning on heterogeneous systems. `https://www.tensorflow.org/`, 2015. Software available from tensorflow.org.

[2] T. P. Carvalho, F. A. A. N. Soares, R. Vita, R. d. P. Francisco, J. P. Basto, and S. G. S. Alcalá. A systematic literature review of machine learning methods applied to predictive maintenance. *Computers & Industrial Engineering*, 137:106024, 2019.

[3] V. Chandola, A. Banerjee, and V. Kumar. Anomaly detection: A survey. *ACM Computing Surveys*, 41(3):1–58, 2009.

[4] M. Compare, P. Baraldi, and E. Zio. Challenges to IoT-enabled predictive maintenance for industry 4.0. *IEEE Internet of Things Journal*, 7(5):4585–4597, 2020.

[5] M.-H. Do and N. Vichare. Proactive condition-based maintenance for aircraft landing gear. *Reliability Engineering & System Safety*, 87:155–165, 2004.

[6] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, pages 226–231, 1996.

[7] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2014.

[8] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[9] IEEE. IEEE/ISO/IEC 12207-2017: Systems and software engineering – software life cycle processes. *IEEE Standard*, 2017.

[10] IEEE. IEEE/ISO/IEC 29148-2018: Systems and software engineering – life cycle processes – requirements engineering. *IEEE Standard*, 2018.

[11] F. T. Liu, K. M. Ting, and Z.-H. Zhou. Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008.

[12] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff. LSTM-based encoder-decoder for multi-sensor anomaly detection. *arXiv preprint arXiv:1607.00148*, 2016.

[13] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[14] Y. Ran, X. Zhou, P. Lin, Y. Wen, and R. Deng. A survey of predictive maintenance: Systems, purposes and approaches. *arXiv preprint arXiv:1911.10539*, 2019.

[15] A. Saxena, K. Goebel, D. Simon, and N. Eklund. Damage propagation modeling for aircraft engine run-to-failure simulation. *International Conference on Prognostics and Health Management*, pages 1–9, 2008.

[16] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson. Estimating the support of a high-dimensional distribution. *Neural Computation*, 13(7):1443–1471, 2001.

[17] X.-S. Si, W. Wang, C.-H. Hu, and D.-H. Zhou. Remaining useful life estimation–a review on the statistical data driven approaches. *European Journal of Operational Research*, 213(1):1–14, 2011.

[18] UWE Bristol. Aircraft predictive maintenance automation and optimisation: Synthesised landing gear datasets. UWE Bristol Research Data Repository, 2023.

Appendices

.1 Appendix A: Software Dependencies

- Python 3.13
- pandas $\geq$ 2.0
- numpy $\geq$ 1.24
- scikit-learn $\geq$ 1.3
- TensorFlow $\geq$ 2.14
- Streamlit $\geq$ 1.28
- Plotly $\geq$ 5.15
- SciPy $\geq$ 1.11
- matplotlib $\geq$ 3.7

Total software cost: $0 (all open-source).

.2 Appendix B: Key Code Listings

```python
def isolation_forest_detect(
    df: pd.DataFrame, columns: list[str],
    contamination: float = 0.05
) -> pd.DataFrame:
    result = df.copy()
    scaler = StandardScaler()
    X = scaler.fit_transform(df[columns])
    model = IsolationForest(
        contamination=contamination,
        random_state=42, n_jobs=-1
    )
    preds = model.fit_predict(X)
    result["is_anomaly"] = preds == -1
    result["anomaly_score"] = model.decision_function(X)
```

```
    return result
```

Listing 1: Isolation Forest Anomaly Detection

```
def estimate_rul(series: pd.Series,
                 critical_threshold: float) -> dict:
    x = np.arange(len(series))
    slope, intercept = np.polyfit(x, series, 1)
    if slope >= 0:
        return {"rul": None, "slope": slope,
                "message": "No decline detected"}
    rul_step = (critical_threshold - intercept) / slope
    remaining = max(0, rul_step - len(series))
    return {
        "rul": remaining,
        "slope": slope,
        "intercept": intercept,
        "critical_threshold": critical_threshold,
        "predicted_failure_step": rul_step,
    }
```

Listing 2: RUL Estimation via Linear Extrapolation

.3 Appendix C: Dataset Summary

Table 1: Summary of the Six Landing Gear Datasets

ID	System	Sets	Length	Sensors
1	Bogie Pitch Trimmer (Hydraulics)	200	200	8
2	Tyre Pressure (Pressure Drop)	800	50	8
3	Tyre Pressure (Temp During Landing)	600	180	2
4	Brakes (Max Temperature)	100	500	8
5	Brakes (Deceleration During Takeoff)	200	100	24
6	Brakes (Deceleration Over Cycles)	400	50	24

.4 Appendix D: User Manual

To run the system:

1. Navigate to the project directory: `cd CE430`
2. Activate the virtual environment: `source venv/Scripts/activate`
3. Launch the dashboard: `streamlit run app.py`

4. Open `http://localhost:8501` in a web browser.
5. Use the sidebar to navigate between pages and configure analysis parameters.